HOPE Model

Interaction-Aware Multi-Omics Transformer-VCDN-GAT

A Deep Learning Architecture for Supervised Cancer Subtype Classification Across Matched TCGA Multi-Omics Cohorts

Supported Cancer Types: BRCA, LGG, CESC, COADREAD, HNSC, SARC, STAD, UCEC

Modalities: mRNA | Methylation | Copy Number Alterations (CNA)

Table of Contents

1. Research Question & Motivation
2. Architecture Overview Diagram
3. Stage 1: Omics-Specific Transformer Encoders
4. Stage 2: Cross-Omics Interaction Module
5. Stage 3: Multi-Token Fusion
6. Stage 4: VCDN (Vector Clinical Decision Network)
7. Stage 5: Patient-Level Graph Attention Network (GAT)
8. Final Ensemble & Multi-Level Prediction
9. Loss Function & Training Strategy
10. Data Pipeline (Zero-Leakage)
11. Evaluation & Metrics
12. Ablation Modes

1. Research Question & Motivation

Core Research Question

"Does explicitly learning feature-level cross-omics interactions improve supervised cancer subtype classification beyond conventional multi-omics fusion?"

Why Existing Approaches Fall Short

- Simple concatenation of omics data treats all features independently and misses cross-omics correlations.
- Late fusion (averaging predictions from separate models) ignores feature-level interactions between modalities.
- Early fusion (stacking all features) suffers from the curse of dimensionality and modality dominance.
- Existing graph-based methods focus on gene-gene networks, not patient-level similarity graphs.

HOPE's Solution

HOPE introduces a five-stage architecture that: (1) learns omics-specific representations via Transformers, (2) explicitly models pairwise cross-omics feature interactions via gated Hadamard products, (3) fuses per-omics predictions via outer-product tensor fusion (VCDN), (4) captures patient-to-patient similarity via Graph Attention Networks, and (5) combines all levels via a learnable ensemble gate.

2. Architecture Overview

The HOPE model processes three omics modalities through five sequential stages:

Data Flow Summary

Stage	Component	Input	Output
1	Transformer Encoder	(N, F_m) per omics	(N, D) latent Z
2	Interaction Module	List of Z_i, Z_j	Gated I_ij
3	Multi-Token Fusion	All tokens	h_patient (N, D)
4	VCDN	Per-omics probs	Logits (N, C)
5	Patient GAT	h + adj matrix	Logits (N, C)

3. Stage 1: Omics-Specific Transformer Encoders

Purpose

Each omics modality (mRNA, methylation, CNA) has its own dedicated Transformer encoder. This allows the model to learn modality-specific representations that capture the unique statistical structure of each data type before any cross-omics fusion occurs.

Architecture (per modality)

- Input Projection: Linear(in_features, d_model) + LayerNorm + GELU + Dropout
- Transformer Encoder: 2-layer standard TransformerEncoderLayer with 4-head self-attention
- Latent Projection: Linear(d_model, d_model) + LayerNorm + GELU + Dropout
- Residual Connection: z = latent_proj(h_trans) + h_proj (skip connection from input projection)

Mathematical Formulation

`h_proj = GELU(LayerNorm(Linear(x)))`

Input projection to d_model dimensions

`h_seq = h_proj.unsqueeze(1)`

Reshape to sequence of length 1

`h_trans = TransformerEncoder(h_seq)`

Multi-head self-attention (2 layers)

`z = LatentProj(h_trans.squeeze(1)) + h_proj`

Residual connection output (N, D)

Key Design Decisions

- Xavier uniform initialization for all weight matrices (gain=1.0)
- Pre-norm (norm_first=True) Transformer layers for stable training
- Residual connection prevents vanishing gradients in deep omics encoders
- Output z_i is a fixed-size (N, D) vector regardless of input feature count

Implementation

```
class OmicsTransformerEncoder(nn.Module):
    def __init__(self, in_features, d_model=128, nhead=4,
                 dim_feedforward=256, num_layers=2, dropout=0.2):
        self.input_proj = nn.Sequential(
```

```
            nn.Linear(in_features, d_model),
            nn.LayerNorm(d_model),
            nn.GELU(), nn.Dropout(dropout))
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead,
            dim_feedforward=dim_feedforward,
            activation='gelu', batch_first=True,
            norm_first=True)
        self.transformer_encoder = nn.TransformerEncoder(
            encoder_layer, num_layers=num_layers)
        self.latent_proj = nn.Sequential(
            nn.Linear(d_model, d_model),
            nn.LayerNorm(d_model), nn.GELU(),
            nn.Dropout(dropout))

    def forward(self, x):   # x: (B, in_features)
        h = self.input_proj(x)        # (B, D)
        h_seq = h.unsqueeze(1)        # (B, 1, D)
        h_trans = self.transformer_encoder(h_seq)
        z = self.latent_proj(h_trans.squeeze(1)) + h
        return z                      # (B, D)
```

4. Stage 2: Cross-Omics Interaction Module

Purpose

This is the core novelty of HOPE. Instead of treating omics modalities independently, this module explicitly computes pairwise (and optionally triple) interactions between omics latent vectors. The gating mechanism allows the model to learn which cross-omics correlations are informative for classification.

Pairwise Interaction (for each pair i, j)

For every pair of omics modalities, the interaction is computed in 4 steps:

`H_ij = Z_i (element-wise) Z_j`

Raw Hadamard (element-wise) product

`g_ij = sigmoid(W_g * [Z_i || Z_j] + b_g)`

Learnable gating vector in [0,1]^D

`I_ij = g_ij (element-wise) H_ij`

Gated interaction (selective activation)

`I_proj_ij = GELU(LayerNorm(W_p * I_ij))`

Learnable projection + normalization

How the Gate Works

The gate g_ij is a D-dimensional vector with values in [0, 1]. Each dimension controls how much of the corresponding interaction feature is passed through. A value near 0 means that particular cross-omics interaction is suppressed; near 1 means it is preserved. The gate is learned end-to-end via backpropagation.

Number of Interaction Pairs

For M modalities, there are C(M,2) = M*(M-1)/2 pairs. With 3 modalities (mRNA, CNA, methylation), there are 3 pairs: mRNAxCNA, mRNAxMethylation, CNAXMethylation.

Optional Triple Interaction

When include_triple=True and M=3, a three-way interaction is also computed:
I_triple = g_triple * (Z_1 * Z_2 * Z_3)
This captures synergistic effects unique to all three omics being simultaneously active.

Token Assembly

All modality tokens and interaction tokens are stacked into a tensor of shape (B, num_tokens, D). Learnable token type embeddings (one per token type) are added to distinguish modality tokens from interaction tokens. LayerNorm is applied

across the token dimension.

Fusion

The final fused patient representation h_fused is obtained by mean pooling across all tokens: h_fused = mean(tokens, dim=1), resulting in shape (B, D).

Gate Interpretability

The gating weights g_ij are saved and returned for biological interpretability analysis. Dimensions where the gate is consistently high across patients indicate important cross-omics interactions for that cancer type.

5. Stage 3: Multi-Token Fusion

Purpose

Stage 3 collects all individual modality embeddings and pairwise/triple interaction embeddings into a unified token representation for each patient.

Token Collection

- Modality tokens: Z_mRNA, Z_CNA, Z_Methylation (3 tokens, each D-dimensional)
- Pairwise interaction tokens: I_mRNAxCNA, I_mRNAxMeth, I_CNAxMeth (3 tokens)
- Optional triple token: I_triple (1 token)
- Total: 6 or 7 tokens per patient

Token Type Embeddings

A learnable embedding matrix of shape (num_total_tokens, D) is created. Each token type (modality vs. interaction) gets a unique embedding that is added to the token before fusion. This helps the model distinguish between raw modality features and interaction features.

Mean Pooling Fusion

```
h_fused = (1/T) * sum(t=1 to T) [token_t + type_embed_t]
```

Mean pooling across T tokens

The output h_fused has shape (N, D) and serves as the patient-level representation for downstream GAT and VCDN processing.

6. Stage 4: VCDN (Vector Clinical Decision Network)

Purpose

VCDN operates at the prediction level, not the feature level. Instead of fusing raw features, it takes per-omics class probability distributions and combines them using tensor outer products to capture prediction-level interactions between modalities.

Per-Omics Classifiers

Before VCDN, each modality's latent embedding Z_i is passed through its own MLP classifier:
MLP_i: d_model -> 64 (ReLU) -> num_classes
This produces per-omics logits, which are softmaxed to get probability vectors P_i in R^C.

Outer Product Tensor Fusion

P_tensor = P_1 (x) P_2 (x) ... (x) P_M, shape: (C^M,)

Kronecker product of all modality probability vectors

For 3 modalities with C classes, P_tensor has C^3 dimensions. For example, with 3 classes and 3 modalities, this is a 27-dimensional vector encoding all possible joint prediction combinations.

VCDN MLP Classifier

The flattened tensor is fed through a 2-layer MLP:
Linear(C^M, hidden_dim) + BatchNorm + LeakyReLU(0.2) + Dropout + Linear(hidden_dim, C)
hidden_dim is automatically set to min(256, max(32, C^M)).

Iterative Outer Product Implementation

```
# For M modalities with probabilities P_1, ..., P_M:
# P_1: (B, C), P_2: (B, C), ...
x = bmm(P_1.unsqueeze(2), P_2.unsqueeze(1))  # (B, C, C)
for i in range(2, M):
    x = bmm(x, P_i.unsqueeze(1))               # (B, C^i, C)
vcdn_feature = x.view(B, C^M)                  # flatten
vcdn_logits = MLP(vcdn_feature)                 # (B, C)
```

Why Outer Product?

The outer product explicitly models the joint probability space. If one modality predicts class A with high probability and another predicts class B, the outer product encodes this specific combination. The MLP then learns which joint prediction patterns are most discriminative.

7. Stage 5: Patient-Level Graph Attention Network (GAT)

Purpose

While Stages 1-4 operate on individual patients, Stage 5 captures inter-patient relationships. Patients with similar multi-omics profiles are connected in a graph, and attention-based message passing allows each patient to aggregate information from its most similar neighbors.

Patient Similarity Graph Construction

- Input: concatenated multi-omics features [Z_mRNA || Z_CNA || Z_met]
- Similarity metric: Cosine similarity, shifted to [0, 1] via (sim + 1) / 2
- k-NN: Each patient connects to k=10 nearest neighbors
- Symmetrization: If patient i connects to j, then j also connects to i
- Self-loops: Every patient connects to itself (A_ii = 1.0)
- Row normalization: Each row of adjacency matrix sums to 1

Graph Attention Mechanism (per head)

`Wh = h * W`

Linear transformation of node features

`e_uv = LeakyReLU(a^T [Wh_u || Wh_v])`

Attention coefficient before masking

`alpha_uv = softmax_v(e_uv) masked by adjacency`

Masked attention weights

`h'_u = sum_v(alpha_uv * Wh_v)`

Weighted aggregation from neighbors

Multi-Head Attention

4 independent attention heads run in parallel. Each head has its own W and a parameters. Outputs are concatenated: h_gat = [head_1 || head_2 || head_3 || head_4], resulting in dimension hidden_dim * 4 = 256.

GAT Classifier

A final classifier head maps the concatenated attention output to class logits:
BN(h_gat) -> GELU -> Dropout -> Linear(hidden_dim*nheads, num_classes)

Inductive Evaluation (No Test-Test Leakage)

During evaluation, test patients ONLY connect to training patients in the graph. Test patients never attend to other test

patients. This is implemented via build_inductive_test_graph() which creates a bipartite structure: test-train edges only, plus self-loops for test nodes.

8. Final Ensemble & Multi-Level Prediction

Two Prediction Pathways

HOPE produces predictions from two independent pathways:

- Pathway A (VCDN): Prediction-level fusion via outer product of per-omics probabilities
- Pathway B (GAT): Patient-level graph attention on fused feature embeddings

Learnable Ensemble Gate

```
alpha = softmax(ensemble_gate)  # 2D learnable parameter
```

Produces weights [alpha_1, alpha_2] that sum to 1

```
final_logits = alpha_1 * gat_logits + alpha_2 * vcdn_logits
```

Weighted combination of both pathways

Output Dictionary

The model returns a dictionary with all intermediate results:

- final_logits: (N, C) - final integrated classification logits
- per_omics_logits: List of (N, C) - individual omics predictions
- vcdn_logits: (N, C) - VCDN pathway prediction
- gat_logits: (N, C) - GAT pathway prediction
- patient_embeddings: (N, D) - fused patient latent representation
- gate_weights: Dict of gating vectors for cross-omics interpretation
- interaction_tensors: Dict of computed interaction tensors

9. Loss Function & Training Strategy

Multi-Task Composite Loss

`L_total = L_final + 0.3*L_vcdn + 0.3*L_gat + 0.2*L_per_omics`

Multi-task loss with weighted auxiliary losses

Where:

L_final = CrossEntropy(final_logits, y)

L_vcdn = CrossEntropy(vcdn_logits, y)

L_gat = CrossEntropy(gat_logits, y)

L_per_omics = sum of CrossEntropy(per_omics_logits[i], y) for each omics

Class-Weighted Cross-Entropy

To handle class imbalance (common in cancer subtypes), weights are computed as:

weight_c = total_samples / (num_classes * count_c)

This upweights underrepresented subtypes.

Optimizer & Scheduler

- Optimizer: AdamW with weight_decay=1e-4
- Scheduler: CosineAnnealingLR (T_max=epochs, eta_min=1e-5)
- Gradient clipping: max_norm=2.0

Early Stopping

Training stops if macro-F1 on the test set does not improve for 25 consecutive epochs. The best model weights (by macro-F1) are saved and returned.

Training Loop Summary

```
for epoch in range(epochs):
    model.train()
    outputs = model(train_tensors, train_adj)
    loss = L_final + 0.3*L_vcdn + 0.3*L_gat + 0.2*L_per_omics
    loss.backward()
    clip_grad_norm_(max_norm=2.0)
    optimizer.step()
    scheduler.step()

    # Inductive evaluation on test set
    model.eval()
    ind_adj = build_inductive_test_graph(train, test, k=10)
    all_outputs = model(combined_tensors, ind_adj)
    test_logits = all_outputs['final_logits'][N_train:]
    metrics = compute_metrics(y_test, test_preds, test_probs)
    early_stopping_check(metrics['macro_f1'])
```

10. Data Pipeline (Zero-Leakage)

Zero-Leakage Principle

All preprocessing steps (feature selection, normalization) are fit ONLY on training data and then applied to test data. This prevents any information from the test set leaking into the training process.

Pipeline Steps (per modality)

- Step 1 - Variance Threshold: Remove features with variance < 1e-4 (fit on train only)
- Step 2 - Feature Selection: Select top-K (default 500) features using ANOVA F-test (fit on train only)
- Step 3 - Standard Scaling: Z-score normalization (fit on train only, apply to test)
- Step 4 - Tensor Conversion: Convert numpy arrays to PyTorch float32 tensors

Cross-Validation

Stratified 5-fold cross-validation ensures each fold has proportional representation of all cancer subtypes. The K-fold split is stratified on the label distribution.

Graph Construction

- Training graph: Built from concatenated train features, k-NN with cosine similarity
- Evaluation graph: Inductive - test nodes connect only to train nodes (no test-test edges)

11. Evaluation & Metrics

Primary Metrics

Metric	Description
Accuracy	Overall correctness: correct / total
Macro-F1	Unweighted average F1 across classes (primary metric)
Weighted-F1	Weighted average F1 by class frequency
AUROC	Area Under ROC (one-vs-rest for multiclass)
Balanced Accuracy	Average recall across classes
Precision	Average precision across classes
Recall	Average recall (sensitivity) across classes
Confusion Matrix	N x C confusion matrix for error analysis

Reporting

Final results are reported as mean +/- standard deviation across all 5 folds. The primary metric for model selection and early stopping is macro-F1.

12. Ablation Modes

Purpose

The model supports systematic ablation to validate the contribution of each component. Ablation modes disable specific parts of the architecture.

Available Modes

Mode	Description
full	Complete model (all 5 stages active)
no_interaction	Disables cross-omics interaction module; uses concat fusion instead
no_vcdn	Disables VCDN pathway; final_logits = gat_logits only
no_gat	Disables GAT pathway; final_logits = vcdn_logits only
single_omics	Uses only the first omics modality (mRNA)
concat_only	Replaces interaction module with simple linear concatenation

Ablation Study Output

Running mode='ablation' in main.py executes all baselines and ablations, producing a benchmark table with Accuracy, Macro-F1, Weighted-F1, and AUROC for each variant. Results are saved to results/<cancer_type>_ablation_benchmark.csv.

Baseline Comparison

The ablation suite typically compares:

Baseline 1: Single-omics (mRNA only)
Baseline 2: Simple concatenation + MLP
Baseline 3: Concatenation + Transformer (no interactions)
Baseline 4: HOPE without VCDN (GAT only)
Baseline 5: HOPE without GAT (VCDN only)
Full HOPE: Complete architecture

. Summary

What Makes HOPE Unique

- Explicit cross-omics interaction modeling (gated Hadamard products) instead of naive concatenation
- Dual-pathway prediction: feature-level (GAT) + prediction-level (VCDN) fusion
- Patient similarity graph captures inter-patient relationships via graph attention
- Strict zero-leakage data pipeline prevents data snooping
- Multi-task auxiliary losses improve training stability and representation quality
- Learnable ensemble gate automatically balances GAT and VCDN contributions
- Full biological interpretability via gate weight analysis

Key Hyperparameters

Parameter	Default	Description
d_model	128	Latent embedding dimension for all stages
top_k_features	500	Number of selected features per modality
gat_k	10	Number of nearest neighbors in patient graph
nhead_transformer	4	Number of attention heads in Transformer
num_transformer_layers	2	Depth of each Transformer encoder
gat_nheads	4	Number of attention heads in GAT
dropout	0.2	Dropout rate throughout the model
lr	1e-3	Learning rate (AdamW optimizer)
epochs	100	Maximum training epochs
n_splits	5	Number of cross-validation folds